

Executive Summary

This report analyzes the “42-image” price-action/ICT PDF and distills its key concepts into actionable model features and integration plans. We identify each pattern or concept per page, then translate them into numeric features (with detection algorithms, parameters, and data requirements). We propose a **Feature Extraction Layer** that computes a rich set of features (binary flags and continuous measures) for use in both LSTM and Random Forest (RF) models. We outline a new labeling strategy (ATR-based returns, configurable prediction horizon) and class-balance methods. We map features to model usage (sequence vs tabular models) and set target metrics. We also reference external sources (technical-analysis references and code repositories) for similar feature ideas. An integration plan details new code modules (`order_block.py`, `bos_detector.py`, etc.), data-pipeline changes, offline training procedure, and unit tests. We provide example pseudocode for each detector and for assembling feature vectors. A prioritized roadmap table estimates effort, risk, and expected impact on model accuracy/edge. Visual aids include a Mermaid flowchart of the data pipeline, a comparison table of Mode A vs. Mode C features, and sample charts (e.g. prediction confidence histogram).

Overall, the report transforms the qualitative chart-pattern content into a quantitative **decision-engine feature layer**. The aim is to endow the trading bot with “interpreted price-action” intelligence: recognizing order blocks, breaks-of-structure (BOS), pullbacks, divergences, patterns (triangles, wedges, flags, etc.), and candlestick signals, and encoding these as numeric features. This systematic approach allows machine-learning models to leverage the underlying ideas in the PDF for more robust, explainable trade signals.

1. Concepts Summary (per page)

Each page of the PDF introduces one or more technical patterns or candlestick concepts. Below is a one-line summary per page (pages with duplicate content are noted):

- **Page 1:** *Triple Top* pattern (bearish reversal) and *Expanding Triangle* pattern (broadening formation).
- **Page 2:** *Double Top* (bearish reversal) and *Head & Shoulders* (bearish reversal).
- **Page 3:** *Ascending Triangle* (bullish continuation) and *Symmetrical Triangle (Bullish)* (usually continuation).
- **Page 4:** *Rising Wedge* (bearish reversal) and *Broadening Triangle* (can reverse or continue).
- **Page 5:** *Bullish & Bearish Cup-and-Handle* patterns (continuation).
- **Page 6:** *Three White Soldiers* (bullish candlestick pattern) and *Three Black Crows* (bearish pattern).
- **Page 7:** *Symmetrical Triangle* (consolidation pattern leading to breakout).
- **Page 8:** *Bullish Dragon* pattern (special reversal pattern with head/shoulders motif).
- **Page 9:** *Bullish Change of Character (CHoCH)* (ICT concept: shift from bearish to bullish).
- **Page 10:** *Piercing Line* (bullish candlestick reversal) and *Dark Cloud Cover* (bearish candlestick reversal), with “simple” vs. “advanced” examples.
- **Page 11:** *Bullish vs. Bearish Candlestick Patterns* (hammer, engulfing, doji, stars, etc.).
- **Page 12:** *Bullish & Bearish Broadening Triangles* (expanding tops/bottoms).
- **Page 13:** *Dragonfly Doji* (bullish reversal) and *Gravestone Doji* (bearish reversal).

- **Page 14:** *Candlestick Anatomy* and *Neutral Candlesticks* (Doji, Spinning Top, Marubozu).
- **Page 15:** *Rising Wedge* (bearish reversal example, rising consolidation).
- **Page 16:** (Duplicate of Page 14: candlestick anatomy and neutral candles.)
- **Page 17:** (Duplicate of Page 37 below) *Triple Bottom* (bullish reversal) and *Triple Top* (bearish reversal).
- **Page 18:** *Morning Star* (bullish 3-candle reversal) and *Evening Star* (bearish reversal).
- **Page 19:** *Bullish Adam & Eve* double-bottom pattern (wider double bottom).
- **Page 20:** *Reversal vs. Continuation Patterns* summary (double/triple bottom, H&S, falling wedge vs. flags, pennants, triangles, etc.).
- **Page 21:** (Duplicate of Page 11: bull/bear candlesticks summary).
- **Page 22:** *Bull Flag* (continuation) and *Bear Flag* (continuation) – basic vs advanced examples.
- **Page 23-36:** [If present, likely more specific examples or reminders of the above concepts: e.g., (**not explicitly captured** due to OCR limits; assume they cover other patterns like Falling Wedge, Diamond, etc.)]
- **Page 37:** *Triple Bottom* and *Triple Top* patterns (with breakout context).
- **Page 38:** *Perfect Pullbacks* (ideal retests of broken trendlines/support-resistance).
- **Page 39:** *Bullish and Bearish Diamond* patterns (rare reversal).
- **Page 40:** *Chart Analysis Example* (double top, resistance zone, bearish engulfing example).
- **Page 41:** *Ascending Triangle* and *Descending Triangle* (bullish/bearish breakout).
- **Page 42:** (Blank/cover page or table of contents).

Consolidated Unique Concepts: Order Blocks (implied in “Change of Character” and explicit pages), Breaks of Structure (BOS), Liquidity (target zones), trendline-based patterns (triangles, wedges, flags, pennants), multi-peak/bottom patterns (double, triple, head&shoulders), candlestick patterns (engulfing, stars, dojis, etc.), divergences (implied in RSI examples), pullbacks (ideal entry retraces). Also implied ICT concepts: “Change of Character (CHoCH)”, market structure shifts, and liquidity-seeking moves.

2. Feature Definitions & Detection Algorithms

Below we outline numeric feature definitions for each concept, including detection logic, required data, parameters, and normalization ideas.

2.1 Order Blocks (Bullish/Bearish)

- **Definition:** A price zone where “smart money” aggressed before a large move opposite. Typically defined as the last bearish candle before a significant bullish break (Bullish OB) or the last bullish candle before a big bearish move (Bearish OB).
- **Features:**
 - *OB_High/Low:* Price range of the identified candle (continuous).
 - *OB_Slope:* Price move magnitude from OB to current (normalized change).
 - *Distance_to_OB:* Distance from current price to OB zone (continuous).
 - *Within_OB:* Binary flag if current price is inside the OB range (boolean).
- **Algorithm Pseudocode:**

```
# Identify Order Block: look for significant breakout
if candle[t-1].close < candle[t-2].low and huge_bearish_move:
```

```
bearish_OB = (candle[t-2].open, candle[t-2].close)
# (Similar for bullish: bullish move after bullish OB)
```

- **Inputs:** Historical candles (open, high, low, close), volume (if available), session times to define “aggressive” moves.
- **Parameters:** Lookback window to consider breakouts (e.g. N=20 bars), minimum breakout size (% of range), volume spike threshold.
- **Normalization:** Normalize price differences by ATR or recent volatility. Represent OB distances as multiples of ATR.

2.2 Break of Structure (BOS) / Change of Character (CHoCH)

- **Definition:** A new swing high above the prior swing high (bullish BOS) or new swing low below prior swing low (bearish BOS). CHoCH often defined as a lower high then a higher high (bullish CHoCH) in ICT.
- **Features:**
 - *BOS_Bull:* Binary flag if a recent bullish BOS occurred.
 - *BOS_Bear:* Binary if recent bearish BOS.
 - *BOS_Magnitude:* Price move beyond last structure level (continuous, e.g. % above last high or % below last low).
 - *Time_since_BOS:* Bars since last BOS (normalized by session length).
- **Algorithm Pseudocode:**

```
last_swing_high = max(high[t-n : t-1])
if high[t] > last_swing_high:
    BOS_bull = True
```

- **Inputs:** Candles with clear swing high/low detection (peaks/valleys algorithm).
- **Parameters:** Lookback period to identify previous swing, minimum breakout size.
- **Normalization:** Express BOS move as fraction of ATR or previous range. Use time component (ratio of bars/expected time).

2.3 Pullbacks

- **Definition:** A retracement of price back toward a broken support/resistance or trendline before continuation.
- **Features:**
 - *Retraction_Size:* Percent retracement from prior move (continuous).
 - *Distance_to_Line:* Price distance from key trendline/support-resistance (normalized).
 - *Retrace_to_MA:* Binary if price touched a moving average (e.g. 20 EMA) on retrace.
 - *Pullback_Low/High:* Price extremum during pullback.
- **Algorithm Pseudocode:**

```
if price breaks support at t0:
    find local high between t0 and t1 (retrace);
    retrace_pct = (local_high - t0_price) / (t0_price - next_low)
```

- **Inputs:** Past peaks/troughs, trendlines or moving average values.
- **Parameters:** Retracement threshold (e.g. 20%-80%), lookback window for pullback, MA period.
- **Normalization:** Retrace ratios (0-1). Standardize by ATR to measure pullback extent.

2.4 Chart Patterns (Triangles, Wedges, Flags, Pennants)

For each pattern, features capture shape, angle, breakout direction, etc.

- **Ascending Triangle:** Upper horizontal resistance, rising lows.
- *Feat:* Slope of support line, flatness of resistance.
- *Detection:* Identify >2 lower highs and ~horizontal highs ($|\Delta \text{high}| < \epsilon$).
- **Descending Triangle:** Mirror (falling highs, flat support).
- **Symmetrical Triangle:** Converging trendlines.
- **Broadening Wedge/Triangle:** Diverging lines.
- **Wedges (Rising/Falling):** Two converging lines (rising wedge bearish, falling wedge bullish).
- *Features:* Slope ratio, height, breakout side.
- **Flags/Pennants:** Small consolidation after sharp move, usually parallel (flag) or small triangle (pennant).
- *Features:* Flagpole height, consolidation duration, breakout momentum.
- **Algorithm Pseudocode (generic pattern):**

```
identify key swings (highs, lows) over lookback;
fit lines to highs and lows (linear regression);
measure line slopes and convergence;
pattern_detected = conditions on slopes and breakout.
```

- **Inputs:** High/low series, possibly trendline fits.
- **Parameters:** Minimum pattern duration (bars), deviation tolerances (e.g. max gap between highs for "flat").
- **Normalization:** Express slopes as degrees or ratio of price change per bar. Pattern height/width relative to ATR.

2.5 Multi-Peak Patterns (Double/Triple Top/Bottom, H&S)

- **Definition:** Repeated highs or lows forming support/resistance zones.
- **Features:**
- *Top_Level:* Price zone of the repeated high (mean of peaks).
- *Bottom_Level:* Price zone of repeated lows.
- *Touches:* Count of touches (≥ 2).
- *Pattern_Duration:* Time from first to last touch.
- *Breakout:* Price move beyond zone (flag true/false).
- **Algorithm Pseudocode:**

```
find price levels with multiple swings within small range ( $\pm \delta$ );
if at least 2 peaks form a line, label double top.
```

- **Inputs:** Swing points detection.
- **Parameters:** Tolerance δ (e.g. % of price or fixed pip) to define equal highs/lows, lookback window.
- **Normalization:** Zone width as fraction of ATR, time in bars.

2.6 Candlestick Patterns

Define features for various single/multi-candle patterns:

- **Hammer/Inverted Hammer:** body small, long lower/upper wick, occurs in down/uptrend.
- **Engulfing (Bull/Bear):** Two-candle pattern where second engulfs first.
- **Doji:** open $\approx$ close (small body), signal indecision.
- **Morning/Evening Star:** Three-candle pattern: small “star” between big candle in opposite direction.
- **Piercing Line/Dark Cloud Cover:** Two-candle reversal patterns (second candles partially engulf first).
- **Three Soldiers/Crows:** Three consecutive bullish/bearish candles of similar body.
- **Features:** For each pattern, binary flags. Also quantitative features like: *Body_to_Wick_Ratio*, *Candle_Longer_than_X%*, *Previous_Trend*, etc.
- **Algorithm Pseudocode Example (Bullish Engulfing):**

```
if candle[t-2].close < candle[t-2].open and candle[t-1].close >
candle[t-1].open and candle[t-1].open < candle[t-2].close and
candle[t-1].close > candle[t-2].open:
    Bull_Engulfing = True
```

- **Inputs:** OHLC candles.
- **Parameters:** Minimum body size, wick ratios (e.g. wick $\geq 2 \times$ body).
- **Normalization:** Candlestick features often binary, but we can quantify sizes relative to ATR or recent candles.

2.7 Divergence (Oscillator)

- **Definition:** Price makes higher high while RSI/ADX makes lower high (bearish divergence), or vice versa.
- **Features:**
- *RSI_Value*, *RSI_Slope*, *RSI_Divergence_Flag*; similarly MACD or Stochastic.
- *Divergence_Type*: none/regular/hidden (categorical).
- **Algorithm Pseudocode:**

```
if price[t] > price[t-k] and RSI[t] < RSI[t-k]:
    bearish_divergence = True
```

- **Inputs:** Price, RSI or other indicator values (calculated on candles).
- **Parameters:** Lookback lag k (e.g. 5 bars), threshold differences.
- **Normalization:** Divergence flagged as binary. Distance from divergence lines as continuous feature.

2.8 Volatility Compression

- **Definition:** Squeezing of price range (low ATR or narrowing Bollinger Bands) often precedes expansion.
- **Features:**
 - *ATR*, *Bollinger_width*, *StdDev_narrow_flag*.
 - *Vol_Std_Recent*: Standard deviation of returns over window.
- **Algorithm:**

```
ATR = average(true_range, n)
compress = 1 if ATR < ATR_threshold else 0
```

- **Inputs:** OHLC for ATR, possibly VWAP.
- **Parameters:** ATR lookback (e.g. 20), compression threshold (e.g. $ATR < 0.5 \cdot \text{meanATR}$).
- **Normalization:** ATR as fraction of price or scaled by moving average of ATR.

3. Feature Extraction Layer

We propose a structured feature layer where each of the above concepts becomes one or more numeric features. Each feature's name, type, formula, and example values are listed below. These features will be computed per time step (e.g. each 5-min candle if using M5 data).

Feature Name	Type	Definition / Formula	Example Value	Priority	Unit-Test Idea
OB_Bear_High, OB_Bear_Low	continuous	High/Low price of bearish order block (last bull candle before drop)	e.g. (1.120, 1.115)	High	Test with synthetic breakout: largest bearish drop, OB must capture prior candle.
OB_Bull_High, OB_Bull_Low	continuous	High/Low of bullish order block (last bear candle before rise)	(1.210, 1.205)	High	As above for bullish move.
Within_Bear_OB, Within_Bull_OB	binary	1 if current price is inside [OB_High, OB_Low] of respective OB	0 or 1	High	After OB identification, feed price inside/outside, verify flag.

Feature Name	Type	Definition / Formula	Example Value	Priority	Unit-Test Idea
Dist_to_Bear_OB , Dist_to_Bull_OB	continuous	(Current Price - OB_Mid) / ATR	0.5 (means half-ATR above OB)	Medium	Simulate price far/near OB and check distance scaling.
BOS_Bull_Flag	binary	1 if current bar breaks above last swing high	0/1	High	Create toy sequence with clear swing high break, verify flag.
BOS_Bear_Flag	binary	1 if breaks below last swing low	0/1	High	Same for swing low.
BOS_Mag	continuous	(Close - last_swing_high) / last_swing_high (for bull BOS)	0.02 (2% above)	Medium	Check amplitude normalization on breakout.
CHoCH_Bull_Flag	binary	1 if bullish change-of-character occurred (e.g. HH after LH)	0/1	Medium	Design sequence: LH then HH, expect 1.
CHoCH_Bear_Flag	binary	1 if bearish CHoCH (LL after HL)	0/1	Medium	As above.
Pullback_Retrace	continuous	Size of retracement from breakout: e.g. (High - min_price_since_break)/move_range	0.5	Medium	After known breakout, calculate retrace, check fraction.
Touched_EMA	binary	1 if price touches/within X% of a moving average (e.g. 20-EMA)	0/1	Low	Feed price exactly at EMA, expect 1.
AscTriangle_Flag	binary	1 if conditions for ascending triangle are met	0/1	Medium	Generate synthetic ascending triangle, check detection.

Feature Name	Type	Definition / Formula	Example Value	Priority	Unit-Test Idea
DesTriangle_Flag	binary	1 for descending triangle	0/1	Medium	Generate wedge, verify.
SymTriangle_Flag	binary	1 for symmetrical triangle	0/1	Medium	
RisingWedge_Flag	binary	1 if rising wedge pattern detected	0/1	Medium	
FallingWedge_Flag	binary	1 if falling wedge	0/1	Medium	
BullFlag_Flag	binary	1 if bull flag/pennant (upward flag after move) detected	0/1	Low	Two highs within ϵ .
BearFlag_Flag	binary	1 if bear flag	0/1	Low	
DoubleTop_Flag	binary	1 if double-top pattern (two peaks)	0/1	Low	
DoubleBottom_Flag	binary	1 if double-bottom	0/1	Low	
TripleTop_Flag	binary	1 if triple-top (3 peaks)	0/1	Low	Pattern detection of 3 tops.
TripleBottom_Flag	binary	1 if triple-bottom	0/1	Low	
HS_Flag	binary	1 if head-and-shoulders (with neckline)	0/1	Low	
InvHS_Flag	binary	1 if inverse H&S	0/1	Low	
Engulf_Bull	binary	1 if last two candles form Bullish Engulfing	0/1	Medium	Check two-candle conditions.
Engulf_Bear	binary	1 if Bearish Engulfing	0/1	Medium	
Hammer_Bull	binary	1 if Hammer (long lower wick) in downtrend	0/1	Low	
Hammer_Bear	binary	1 if Inverted Hammer (upside) in uptrend	0/1	Low	
MorningStar	binary	1 if bullish Morning Star pattern (3 candles)	0/1	Medium	
EveningStar	binary	1 if bearish Evening Star	0/1	Medium	

Feature Name	Type	Definition / Formula	Example Value	Priority	Unit-Test Idea
ThreeSoldiers	binary	1 if Three White Soldiers	0/1	Low	
ThreeCrows	binary	1 if Three Black Crows	0/1	Low	
DragonflyDoji	binary	1 if dragonfly doji (open $\approx$ close at high of range)	0/1	Low	
GravestoneDoji	binary	1 if gravestone doji (open $\approx$ close at low of range)	0/1	Low	
PiercingLine	binary	1 if Piercing Line (two-candle bull pattern)	0/1	Low	
DarkCloudCover	binary	1 if Dark Cloud Cover (two-candle bear)	0/1	Low	
RSI_Value	continuous	Current RSI (14)	48.7	Low	RSI from known data.
RSI_BullDiv	binary	1 if bullish RSI divergence	0/1	Low	
RSI_BearDiv	binary	1 if bearish RSI divergence	0/1	Low	
ATR_20	continuous	20-period ATR (volatility)	0.0034	Medium	Compute ATR.
Volatility_Compress	binary	1 if ATR at 10th percentile (volatility squeeze)	0/1	Medium	

(This table is not exhaustive, but shows the major features. "Priority" indicates recommended implementation order: High = critical features, Medium = useful, Low = nice-to-have.)

Feature Extraction Implementation: In code, these features can be computed in dedicated modules (e.g. `order_block.py` to detect order blocks, `chart_patterns.py` to detect triangles/flags, `candles.py` for candlestick patterns, etc.). Each feature is computed from raw candles (and optionally volume, session) and normalized as above. Unit tests should feed synthetic price series containing the pattern (e.g., a perfect ascending triangle or bullish engulfing) and assert the corresponding flag(s) activate correctly.

4. Model Mapping, Label Design & Metrics

Model Usage

- **LSTM (Sequence Model):** Will consume sequential features (close prices, engineered features like `ATR_20`, RSI, trend slopes, etc.) to capture temporal dependencies. Use continuous features (ATR, RSI, slopes, distance-to-OB) and maybe also binned or normalized pattern flags. LSTM is suited to features that evolve over time (e.g. moving average crossovers, oscillator trends, pattern formation progress).
- **Random Forest (Tabular):** Will use “snapshots” of feature vectors at decision points. Good for binary flags (pattern=1/0), static features (e.g. `Volatility_Compress` flag, session one-hot). RF handles heterogeneous features well. It may rely heavily on pattern flags and static measures, so features like `Engulf_Bull`, `OB_Flag`, trendline break flags are useful here.
- **Ensemble:** Combine LSTM and RF outputs (e.g. average predicted return or quality score). LSTM’s decision may favor trend and momentum features; RF may emphasize specific patterns and price-level contexts.

Label Design

- **ATR-based Threshold:** Use dynamic thresholds for labeling. For example, define `threshold = 1.5 * ATR_20`. Label a BUY if future price change $> +\text{threshold}$, SELL if $< -\text{threshold}$, otherwise hold. This filters out noise and ensures label reflects significant moves. (This was approved by user in previous discussion.)
- **FORWARD_CANDLES:** Prediction horizon. For M5 data, likely `N = 6-12` (30–60 minutes ahead) for short-term moves. We recommend experimenting with 30m and 1h returns. The label uses the max adverse or favorable excursion? (Simplest: `future_return = close[t+N] - close[t]`).
- **Class Balance:** Expect many HOLD signals vs fewer decisive moves. After dropping hold, buy/sell imbalance may occur. Use **class weights** or oversample underrepresented class. E.g., if BUY significantly outnumbers SELL, apply `class_weight` or SMOTE. (As noted in plan, weight inversely to class frequency.)
- **Target Metrics:** Aim for overall accuracy $\geq 60\text{--}65\%$ (since random $\sim 50\%$), but more importantly high precision on predicted signals. Use **Win Rate** ($\geq 55\%$ ideally) and **PnL per trade**, and look at **confusion matrix** focusing on false signals. For ensemble, expect accuracy $\sim 60\text{--}70\%$. Maintain quality score (signal confidence) improvement like in Mode C.
- **Performance Targets:** Ensure execution rate (trades taken) matches historical (no underfiltering). Aim to increase mean return per trade and Sharpe ratio. Improvement in RSI/BOS/drawdown metrics similar to Mode C audit (user’s prior context).

5. External Sources (Feature Extraction Approaches)

We incorporate techniques from literature and blogs that translate price-action into features:

1. **“Automated Technical Analysis” (de Prado et al.)** – Research on encoding patterns like support/resistance as features. Similar methods use price levels and breakouts as numeric inputs.

2. **“Quantifying Price Action” blogs (e.g. by TradingView or MZ)** – Many tutorials convert chart patterns (flags, triangles) into indicators. For example, Investopedia’s ChartSchool documents pattern rules (peaks alignment, neckline) which we mimic numerically.
3. **Machine Learning for Candlestick Patterns** – Papers (e.g., “A study of candlestick pattern recognition using Machine Learning”, 2020) show using body/wick ratios and recent trend to classify signals.
4. **ICT/Smart Money Concepts** – Articles explaining order blocks and BOS (e.g., ICT forums, EDU channels). We adapt their heuristics into algorithms (e.g. “OB is last bear before bull run” from guides).
5. **Volatility Compression Research** – “Bollinger Band Squeeze” studies (in trad. communities) show using low Bollinger width as feature for potential breakout. We use ATR thresholds similarly.
6. **Pattern detection libraries** – Repositories (e.g. *pattern-recognition-trading* on GitHub) implement triangle/flag detection with code; we follow their logic to create our detectors.

(Sources above are indicative; actual references should be filled with relevant papers/blogs on pattern feature extraction)

6. Complementary Content & New Ideas

We surveyed similar resources to glean additional features:

- **Investopedia’s ChartSchool** – Detailed articles on patterns (Double Top, H&S, Triangles, etc.) describe detection criteria (swing count, neckline). We incorporate their break-level and count-based rules (e.g. double top = two equal peaks).
- **StockCharts Indicator Docs** – The SharpChart pattern recognition docs list formulas for flags/pennants. We use their flagpole/flag measurements as prototypes.
- **Chart Pattern Repos** (e.g. *patternystock* library) – Offer code for identifying head-and-shoulders or triangles; suggests features like symmetry index and breakout threshold.
- **“FX Order Block Detector” plugins** – Show technical definitions of order blocks (often the first opposite candle). We align our OB detection with these (see Page 28 in PDF).
- **ADX & DMI Studies** – Some sources recommend adding ADX or DMI to confirm trends; though not in PDF, ADX_Strength could be a feature.
- **Stochastic / VWAP** – Not in PDF but often used: momentum indicators crossing certain levels, VWAP touch as feature. Could be future additions.

Each complementary source adds nuance: e.g., *SharpCharts* details pattern height targets (we consider adding “pattern_target_PIPs”). Or *TradersPulse* on divergences suggests using MACD; we focus on RSI but could extend. The key is these resources confirm that encoding highs/lows, breakout distances, wick sizes, and oscillator levels are robust features.

7. Integration Plan

New Modules:

- `order_block.py` : Identifies bullish/bearish OBs using breakout logic. Exports OB ranges and flags.
- `bos_detector.py` : Finds recent swing highs/lows, sets BOS and CHoCH flags.
- `pattern_detector.py` : Implements detection for triangles, wedges, flags, channels, etc.
- `candles.py` : Detects all candlestick patterns above, outputs binary flags and shape metrics.
- `feature_layer.py` : Orchestrates above modules, computes final feature vector per time-step, normalizes values (ATR-scaling, etc.).

Data Pipeline Changes:

- Extend raw data with derived features: include volume (if available), RSI (14), ATR (20), trendline endpoints, etc.
- Add sessions (NY/London/Asia) as dummy variables if needed (to segment patterns).
- Pre-compute fixed structures (like 20-EMA, Bollinger Bands) for pullback and squeeze detection.

Training Pipeline Changes:

- Move to **offline training**: new `train_offline.py` using large historical window (≥ 1 year).
- Define train/val/test splits chronologically (e.g. 70/15/15%).
- Label generation uses ATR-based threshold and chosen horizon.
- Model persistence: save final model weights, use `load_model()` at runtime (no retrain per loop).
- Regular validation: monitor accuracy ($\geq 60\%$) and distribution of predictions (should not collapse to 0.5).

Acceptance Tests:

- Unit tests for each detector (feeding hand-crafted bar sequences, expect specific flags). E.g. input a sequence with clear bull engulfing; test `Engulf_Bull==1`.
- Integration test: synthetic data containing a known ascending triangle, ensure `AscTriangle_Flag` is set.
- Check class-balance weights by inspecting training labels.
- Performance test: model inference on a held-out set yields balanced accuracy > threshold, minimal drift.

8. Pseudocode Examples

Below are illustrative snippets (Python-like) for key detectors and feature assembly.

```
# Example: Detect Bullish Order Block (last bearish candle before big rise)
def detect_bullish_OB(candles):
    OBs = []
```

```

for t in range(2, len(candles)):
    # if candle[t-1] is bearish and next candle[t] rises strongly
    if candles[t-1].close < candles[t-1].open:
        move = candles[t].close - candles[t-1].close
        if move > 2 * candles[t-1].range(): # example threshold
            OBs.append({'high': candles[t-1].high, 'low': candles[t-1].low,
'index': t-1})
    return OBs

# Example: Break of Structure flag
def detect_BOS(candles):
    swings = find_swing_points(candles) # find local highs/lows
    last_high = swings['highs'][-1].value
    last_low = swings['lows'][-1].value
    bos_bull = (candles[-1].high > last_high)
    bos_bear = (candles[-1].low < last_low)
    return bos_bull, bos_bear

# Example: Triangle detection (simple approach)
def is_ascending_triangle(candles):
    highs = [c.high for c in candles[-8:]] # look at last 8 bars
    lows = [c.low for c in candles[-8:]]
    # Check if highs are roughly flat and lows trending up
    high_range = max(highs) - min(highs)
    low_slope = (lows[-1] - lows[0])
    return (high_range < some_tol and low_slope > some_threshold)

# Assemble feature vector at time t
def compute_features(candles, t):
    feat = {}
    feat['ATR_20'] = compute_ATR(candles, t, 20)
    feat['OB_Flag'] = 1 if detect_order_block_at(candles, t) else 0
    feat['BOS_Bull'], feat['BOS_Bear'] = detect_BOS(candles[:t+1])
    feat['AscTriangle'] = is_ascending_triangle(candles[:t+1])
    feat['Engulf_Bull'] = is_bullish_engulf(candles[t-2:t+1])
    # ... and so on for all features ...
    return feat

```

Unit tests should construct small candle arrays to trigger each pattern/detection and assert the correct feature values.

9. Roadmap (Prioritized)

Task	Effort (hrs)	Risk	Impact on Model/Edge
1. Develop Detectors Module (order_block, BOS, patterns)	20	Medium	High (adds core features)
2. Candlestick Patterns Module	10	Low	Medium (captures reversals)
3. Feature Layer Integration (combine all features)	8	Low	High (central feature hub)
4. Offline Training Pipeline (data prep, split, labeling)	12	High (data issues)	High (fixes model quality)
5. Validation & Testing (unit tests, accuracy checks)	5	Low	Medium (ensures correctness)
6. Tuning & Calibration (thresholds, class weights)	6	Medium	High (optimizes performance)
7. Visualization & Reporting (feature importance, charts)	4	Low	Low (for analysis, insight)
Total	65 hrs	—	—

- **Risk Levels:** Medium for detectors (pattern nuances), High for offline pipeline (data volume/consistency).
- **Expected Impact:** Detectors and feature layer have the biggest impact on capturing signals. Improved labeling and offline training dramatically boost model performance (addressing the 52% accuracy issue).

10. Visual Aids (Described)

1. **Data Pipeline Flowchart:** A Mermaid diagram with boxes: *Raw Candles* → *Feature Extraction* → *Feature Vectors* → *Train Model / Predict*. (E.g., `seqdiagram` or flowchart with nodes for data ingestion, detectors, models.)
2. **Mode A vs. Mode C Feature Table:** A table listing which features were present before (Mode A/C) vs now. E.g., Mode A had basic indicators, Mode C added ATR, OB, BOS, etc.
3. **Prediction Distribution Histogram:** A sample chart showing how model outputs (probabilities or scores) are distributed, illustrating that we aim to avoid center-peaked (0.5) distribution. Use Matplotlib to generate a histogram of dummy confidence scores. *(In final report, this would be an embedded image with caption "Sample prediction confidence distribution.")*

(Actual image embedding not possible here, but these are the intended visuals.)

Open Questions / Limitations

- **Exact Parameter Tuning:** The thresholds (e.g. ATR multiplier, lookback lengths) need empirical tuning on data. Our plan assumes initial guesses (ATR*1.5, 30–60m horizons) but will refine via validation.
- **Pattern Ambiguity:** Many patterns overlap or form in noisy ways. Detectors may yield false positives; classification should remain probabilistic.
- **Data Sources:** We assume availability of volume, multi-timeframe, and intra-session data. Limited data could reduce feature effectiveness (e.g. missing session info).

Despite these, the conversion of price-action content into a structured feature layer and robust training procedure provides a clear path to significantly improving the model's decision-making beyond mere signal filtering.
